

AI BlogNest API

AI-Powered Blog Management and Content Generation Platform

Team ID: 6ab4ccd16f290be8a6b460f4

Project Name: aiblognest

Date: 15 March 2026

Project Description

AI BlogNest is a RESTful backend built with Node.js and Express. It uses MongoDB with Mongoose for persistence, JWT authentication and bcrypt for user security, and Gemini-powered AI services for blog content generation and summarization. The documented system follows an MVC architecture and exposes REST API endpoints for authentication, blog CRUD operations, and AI-assisted content workflows.

Core Capabilities

- User registration and login.
- JWT-protected routes and profile access.
- Blog creation, reading, updating, and deletion.
- AI-powered blog content generation.
- AI-powered blog summarization.
- REST API testing through Thunder Client/Postman.

Technical Architecture

The supplied project documentation describes a backend-oriented REST architecture. Client requests are received by Express routes, processed by application logic/controllers, persisted through Mongoose/MongoDB where applicable, and integrated with Gemini for AI generation and summarization.

Layer	AI BlogNest implementation
Client / API Consumer	Thunder Client, Postman, or another HTTP client
Web/API Layer	Node.js + Express.js
Application Structure	MVC architecture
Authentication	JWT-based authentication
Password Security	bcrypt / bcryptjs
Database	MongoDB accessed through Mongoose
AI Integration	Google Generative AI / Gemini
Configuration	dotenv environment variables
Development	nodemon for development execution

Architecture flow

Client → Express REST endpoint → validation/authentication → controller/service logic → MongoDB/Mongoose and/or Gemini AI → JSON response.

Database note: MongoDB/Mongoose usage is documented, but no ER diagram or complete schema definition was supplied.

Prerequisites

- Node.js runtime and npm.
- MongoDB connection for persistent data storage.
- A Gemini API key for AI generation and summarization.
- Basic JavaScript / Node.js and Express knowledge.
- Thunder Client or Postman for API testing.
- A development environment capable of running npm scripts.

Package configuration

The project package is named **ai-blognest-api**, version 1.0.0, and uses Node.js with Express, Mongoose, JWT, bcryptjs, dotenv, express-validator, morgan, axios, and the Google Generative AI SDK.

Required environment variables

Variable	Purpose
PORT	Port used by the API server
MONGO_URI	MongoDB connection string
JWT_SECRET	Secret used for JWT authentication
GEMINI_API_KEY	Credential for Gemini AI integration
GEMINI_MODEL	Gemini model selection

Project Workflow

Milestone	Activities
1. Architecture & Environment	Configure Node.js/Express project, environment variables, MongoDB, authentication, and Gemini integration.
2. Authentication	Implement registration, login, password hashing, JWT issuance, and protected profile access.
3. Blog Management	Implement create, read, update, and delete blog operations.
4. AI Features	Implement AI blog generation and blog summarization endpoints.
5. API Testing	Use Thunder Client/Postman collection to exercise authentication, blog, and AI endpoints.
6. Run & Deployment Preparation	Use package scripts and environment configuration to run the API locally; deployment details are not supplied.

This workflow is based on the supplied README and API collection. No external deployment result or production hosting information was supplied.

Milestone 1: Architecture and Environment

The first milestone establishes the backend foundation for AI BlogNest.

- Create the Node.js project and install runtime/development dependencies.
- Configure Express as the REST API framework.
- Configure dotenv for environment-based secrets and settings.
- Configure MongoDB connectivity through Mongoose.
- Organize the backend according to the documented MVC architecture.
- Prepare authentication and Gemini integration boundaries.

Application entry point

The package configuration identifies **src/server.js** as the main entry point.

Run commands

```
npm install npm run dev # or npm start
```

Milestone 1 — Environment Configuration

AI BlogNest uses environment variables for runtime configuration and credentials.

Example configuration structure (values intentionally omitted):

```
PORT=5000 MONGO_URI= JWT_SECRET= GEMINI_API_KEY= GEMINI_MODEL=gemini-2.5-flash
```

Secrets should be supplied through the local environment and should not be exposed in documentation or source control. Actual secret values are intentionally not reproduced.

Dependency management

- package.json defines project scripts and dependencies.
- package-lock.json locks the dependency tree for repeatable npm installation.
- nodemon is included as a development dependency.

Milestone 2: Authentication

The authentication module provides account creation, login, and protected profile access.

Endpoint	Method	Purpose
/api/auth/register	POST	Register a new user
/api/auth/login	POST	Authenticate a user and obtain JWT-based access
/api/auth/profile	GET	Access the authenticated user's profile

- Passwords are handled using bcrypt/bcryptjs.
- JWT is used to protect authenticated routes.
- The profile endpoint demonstrates authenticated access.
- Input validation support is included through express-validator.

The supplied API collection includes Register User, Login User, and Get Profile requests.

Milestone 3: Blog Management

AI BlogNest provides a REST CRUD workflow for blog records.

Endpoint	Method	Purpose
/api/blogs	POST	Create a blog
/api/blogs	GET	Retrieve all blogs
/api/blogs/:id	GET	Retrieve one blog by ID
/api/blogs/:id	PUT	Update a blog
/api/blogs/:id	DELETE	Delete a blog

Documented example fields

- title
- content
- category

The supplied API collection includes create, list, get-by-ID, update, and delete requests for the blog module.

Milestone 4: AI Blog Generation

The AI generation module uses Gemini through the Google Generative AI integration documented in the project package and README.

Endpoint	Method	Input	Purpose
/api/ai/generate-blog	POST	topic	Generate AI-assisted blog content from a topic
/api/ai/summarize	POST	content	Generate a summary from supplied blog content

Generation workflow

- Client submits a topic to the AI generation endpoint.
- Backend receives and processes the request.
- Gemini is used to generate blog content.
- The API returns the generated result to the client.

Exact prompt wording and generated output format are not specified in the supplied README, so they are not invented here.

Milestone 4: AI Summarization

The summarization endpoint provides an AI-assisted way to condense supplied blog content.

Endpoint	Method	Input	Purpose
/api/ai/summarize	POST	content	Summarize supplied blog content using the configured Gemini

API collection coverage

The supplied collection contains a request named “Summarize Blog Content”, with a JSON content field. This provides a documented test entry point for the feature.

Limitation: No measured AI response times or sample production responses were supplied.

API Testing and Verification

A Thunder Client/Postman collection is supplied for functional API testing.

Request	Method	Endpoint
Register User	POST	/api/auth/register
Login User	POST	/api/auth/login
Get Profile	GET	/api/auth/profile
Create Blog	POST	/api/blogs
Get All Blogs	GET	/api/blogs
Get Blog By ID	GET	/api/blogs/{{blogId}}
Update Blog	PUT	/api/blogs/{{blogId}}
Delete Blog	DELETE	/api/blogs/{{blogId}}
Generate AI Blog	POST	/api/ai/generate-blog
Summarize Blog Content	POST	/api/ai/summarize

The collection uses environment variables including base_url, token, and blogId. The documented environment sets base_url to http://localhost:5000; token and blogId are populated during testing.

Security and Error Handling

- JWT authentication is used for protected API routes.
- bcryptjs is included for password hashing.
- dotenv is used for environment-based configuration.
- express-validator is included for request validation.
- cors is included for cross-origin request handling.
- morgan is included for HTTP request logging.

Security configuration

Sensitive values such as JWT secrets and Gemini API credentials are represented by environment variables. Actual credentials are intentionally excluded.

Error handling

The supplied documentation identifies a REST API architecture but does not provide a complete error-response specification. Exact status codes and response schemas beyond documented endpoint purposes are therefore not asserted.

Project Structure

The project documentation identifies an MVC architecture and src/server.js as the application entry point.

A complete source-tree listing was not included in the supplied README, so the following is a documentation-level view rather than an invented exact folder tree:

Area	Documented responsibility
src/server.js	Application entry point identified by package.json
MVC components	Controllers, models, routes, and related application modules are part of the documented MVC approach
Environment configuration	.env / environment variables for PORT, MongoDB, JWT, and Gemini settings
API test collection	Thunder Client/Postman collection for functional endpoint testing
package.json	Scripts and dependency definitions
README.md	Setup, environment, endpoint, and testing guidance

Exact filenames for controllers, models, and routes should be copied from the source repository if a source-level submission requires them.

Local Setup and Execution

Step 1 — Configure the environment

Copy `.env.example` to `.env` where available, then provide the required `PORT`, `MONGO_URI`, `JWT_SECRET`, `GEMINI_API_KEY`, and `GEMINI_MODEL` values.

Step 2 — Install dependencies

```
npm install
```

Step 3 — Start development server

```
npm run dev
```

Step 4 — Start production-style Node process

```
npm start
```

Step 5 — Test the API

Import the supplied collection into Thunder Client or Postman, set the base URL, register/login as needed, and execute the documented endpoint requests.

Functional Feature Coverage

Feature	Documented status	API coverage
User Registration	Included	POST /api/auth/register
User Login	Included	POST /api/auth/login
Protected Profile	Included	GET /api/auth/profile
Create Blog	Included	POST /api/blogs
Read Blogs	Included	GET /api/blogs and GET /api/blogs/:id
Update Blog	Included	PUT /api/blogs/:id
Delete Blog	Included	DELETE /api/blogs/:id
AI Blog Generation	Included	POST /api/ai/generate-blog
AI Summarization	Included	POST /api/ai/summarize

These features are directly documented by the supplied README and API collection. Actual execution results are not represented as test evidence in this document.

Testing Approach

The supplied project materials document API testing through Thunder Client/Postman. Functional requests cover authentication, profile access, blog CRUD, AI generation, and summarization.

Test area	Suggested verification
Registration	Submit valid user data and verify successful account creation response.
Login	Submit registered credentials and verify authentication response/token handling.
Profile	Call protected profile endpoint with authentication.
Blog CRUD	Create, list, retrieve, update, and delete a blog.
AI Generation	Submit a topic and verify AI-generated blog response.
Summarization	Submit content and verify a summary response.

Performance evidence

The separate Performance Testing template has been completed with scenarios and target metrics, but actual response-time, throughput, error-rate, CPU, and memory measurements were not supplied. Those values should be entered after an executed performance test.

Deployment and Access Details

Field	Documented value
Local server	Configured through PORT; the documented test environment uses port 5000.
Hosted / deployed URL	Not supplied
Login credentials	Not supplied; credentials should be created during local/API testing.
Hosting provider	Not supplied
Repository link	Not supplied
Demo video link	Not supplied

No public deployment, repository URL, demo credentials, or demo video was included in the supplied project materials. These fields are therefore marked Not supplied.

Environment and Configuration Notes

The project depends on external configuration for MongoDB, JWT, and Gemini services.

Configuration	Requirement
PORT	API server listening port
MONGO_URI	Accessible MongoDB database
JWT_SECRET	Application secret for JWT signing/verification
GEMINI_API_KEY	Valid Google Gemini API credential
GEMINI_MODEL	Configured Gemini model identifier

Configuration safety

- Do not place real API keys or JWT secrets in public documentation.
- Use an environment file locally and keep secret values out of source control.
- Provide an .env.example with placeholder values for evaluator setup where appropriate.

Known Limitations

No.	Known issue / limitation	Workaround / status
1	No public hosted URL was supplied.	Run locally using the documented npm scripts.
2	No repository link was supplied.	Use the submitted project files for evaluation.
3	No measured performance results were supplied.	Execute load/performance scenarios and record actual metrics.
4	No ER diagram/schema document was supplied.	Add the actual MongoDB/Mongoose schema and ER diagram if required.
5	Exact source folder tree is not present in README.	Use the submitted source tree for the authoritative structure.
6	No demo video or screenshots were supplied.	Attach them if required by the assessment.

Future Enhancements

- Add a documented public deployment and repository link.
- Add automated unit/integration tests and publish test results.
- Add an ER diagram and complete database schema documentation.
- Add API response examples and standardized error-response documentation.
- Add performance-test evidence with actual latency, throughput, error, CPU, and memory measurements.
- Add API documentation such as OpenAPI/Swagger if required.
- Expand AI controls such as configurable tone, length, and content format if supported by future implementation.

These are improvement opportunities, not claims that the current implementation already provides these capabilities.

Conclusion

AI BlogNest is documented as a Node.js/Express REST backend combining MongoDB/Mongoose persistence, JWT authentication, bcrypt password protection, and Gemini-powered AI features. Its documented functional scope includes user registration and login, protected profile access, complete blog CRUD operations, AI blog generation, and AI summarization.

The supplied API collection provides a practical test surface for the major documented endpoints. The project is configured to run through npm scripts and environment variables. Deployment URLs, repository details, demo credentials, screenshots, and measured performance results were not supplied and have therefore not been invented.

Final Project Information

Team ID	6ab4ccd16f290be8a6b460f4
Project Name	aiblognest
Project Type	RESTful backend / API
Primary Runtime	Node.js
Framework	Express.js
Database	MongoDB / Mongoose
Authentication	JWT + bcryptjs
AI Integration	Google Generative AI / Gemini